

User Manual

for S32 LIN Driver

Document Number: UM11LINASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	9
3.5 Driver Limitations	10
3.6 Driver usage and configuration tips	10
3.6.1 Frame Timeout Disable Feature	10
3.6.2 Dual Clock Feature	11
3.6.3 How to configure multicore	11
3.7 Runtime errors	13
3.8 Symbolic Names Disclaimer	14
4 Tresos Configuration Plug-in	15
4.1 Module Lin	16
4.2 Container AutosarExt	16
4.3 Parameter LinDisableDemReportErrorStatus	17
4.4 Parameter LinFrameTimeoutDisable	17
4.5 Parameter LinEnableUserModeSupport	18
4.6 Container LinGeneral	19
4.7 Parameter LinMulticoreSupport	19
4.8 Parameter LinDevErrorDetect	20
4.9 Parameter LinIndex	20
4.10 Parameter LinTimeoutMethod	21
4.11 Parameter LinTimeoutDuration	21
4.12 Parameter LinVersionInfoApi	22
4.13 Reference LinEcucPartitionRef	22
4.14 Container LinDemEventParameterRefs	23
4.15 Reference LIN_E_TIMEOUT	23
4.16 Container LinGlobalConfig	24
4.17 Container LinChannel	24
4.18 Parameter LinChannelId	25

4.19	Parameter LinNodeType	25
4.20	Parameter LinChannelBaudRate	26
4.21	Parameter BreakLength	26
4.22	Parameter SlaveSyncBreakLength	27
4.23	Parameter LinResponseTimeout	27
4.24	Parameter LinHeaderTimeout	28
4.25	Parameter LinHwChannel	29
4.26	Parameter LinChannelWakeupSupport	29
4.27	Reference LinClockRef	30
4.28	Reference LinClockRef_Alternate	30
4.29	Reference LinChannelEcuMWakeupSource	30
4.30	Reference LinChannelEcucPartitionRef	31
4.31	Container CommonPublishedInformation	31
4.32	Parameter ArReleaseMajorVersion	32
4.33	Parameter ArReleaseMinorVersion	32
4.34	Parameter ArReleaseRevisionVersion	33
4.35	Parameter ModuleId	33
4.36	Parameter SwMajorVersion	34
4.37	Parameter SwMinorVersion	34
4.38	Parameter SwPatchVersion	35
4.39	Parameter VendorApiInfix	35
4.40	Parameter VendorId	36
5	Module Index	37
5.1	Software Specification	37
6	Module Documentation	38
6.1	LIN Driver	38
6.1.1	Detailed Description	38
6.1.2	Data Structure Documentation	39
6.1.3	Macro Definition Documentation	40
6.1.4	Function Reference	44
6.2	LIN	48
6.2.1	Detailed Description	48
6.3	LIN_IPW	49
6.3.1	Detailed Description	49
6.3.2	Enum Reference	49
6.4	LINFLEXD_IP	50
6.4.1	Detailed Description	50
6.4.2	Data Structure Documentation	51
6.4.3	Macro Definition Documentation	53



6.4.4 Types Reference	56
6.4.5 Enum Reference	56
6.4.6 Function Reference	59



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor AUTOSAR LIN for S32. AUTOSAR LIN driver configuration parameters and deviations from the specification are described in Driver chapter of this document. AUTOSAR LIN driver requirements and APIs are described in the AUTOSAR LIN driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
C/CPP	C and C++ Source Code
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECUM	ECU state Manager
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
LIN	Local Interconnect Network
LSB	Least Significant Bit
MCU	Micro Controller Unit
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of LIN Driver	AUTOSAR Release 4.4.0
2	S32G2 Reference Manual	Rev. 7, February 2023
3	S32G3 Reference Manual	Rev. 3, Feb 2023
4	S32R45 Reference Manual	Rev. 3, 12/2021
4	S32G2: Mask Set Errata for Mask 0P77B	Mask Set Errata for Mask 0P77B v3.0
5	S32G2: Mask Set Errata for Mask 1P77B	Mask Set Errata for Mask 1P77B v1.0
6	S32G3: Mask Set Errata for Mask 1P72B	Mask Set Errata for Mask 1P72B v2.1
7	S32R45: Mask Set Errata for Mask P57D	Mask Set Errata for Mask P57D, Rev. 2.0
8	S32G2 Data Sheet	Rev 6, Dec 2022
9	S32G3 Data Sheet	Rev 2, RC, Feb 2023
10	VR5510 Data Sheet	Rev 5, April 2022
11	S32R45 Data Sheet	Rev. 3, RC — 11/2022

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

3.2 Driver Design Summary

The LIN driver is part of the Real Time Drivers, performs the hardware access and offers a hardware independent API to the upper layer. The only upper layer, which has access to the LIN driver, is the LIN Interface.

A LIN driver can support more than one channel. It means that the LIN driver can handle one or more LIN channels as long as they belong to LIN hardware unit.

This LIN driver supports both Master and Slave mode.

The LIN Driver for S32, uses the LINFLEXD on-chip hardware modules which provides special support for the LIN protocol. It can be used to automate most tasks of a LIN master and slave. It is possible to transmit entire frames (or sequences of frames) and receive data from other LIN node without any CPU intervention.

The LIN physical interface should be connected to the LINFLEXD module pins in order to get the LIN bus voltage levels.

The S32 platform contains up to 3 blocks.

The LINFLEXD hardware unit has the following major features:

- LIN Master Node functionality (master and slave task),
- LIN Slave Node functionality (slave task),
- Compatible with LIN from revisions 1.x and 2.x of the LIN standard,
- Detection of Bit Errors, Physical Bus Errors, Frame Error, Timeout Error and Checksum Errors,
- All status bit can generate maskable interrupts,
- Application layer CRC support,
- Programmable CRC polynom,
- Double Stop Flag insertion after Bit Errors,
- Detection and generation of wakeup characters,
- Programmable wakeup delimiter time,
- Can be configured to include header bits in checksum.

3.3 Hardware Resources

The device family includes S32G274A, S32G254A, S32G233A, S32G234M. S32G399A, S32G398A, S32G379A, S32G378A, S32G338M, S32G339M, S32G358A, S32G359A derivatives and S32R45 derivatives.

The hardware configured by the Lin driver is LINFLEXD.

Lin Physical Channels:

- For all devices of the S32G2/S32G3 family have 3 channels: LINFLEXD_0, LINFLEXD_1, LINFLEXD_2.
- For all devices of the S32R45 family have 2 channels: LINFLEXD_0, LINFLEXD_1.

For example:

In EB tresos, *LINFLEXD_0* channel has named *LINFLEXD_0*, correspondingly as below:



Figure 3.1 Lin hardware channel configuration in EB Tresos.

The LIN channel to microcontroller pin mapping can be done using file *S32G2_IOMUX.xlsx* from S32G Reference Manual or *S32R45_IOMUX.xlsx* from S32R45 Reference Manual or *S32G3_IOMUX.xlsx* from S32G3 Reference Manual.

The RM chapter stated above has a table as below:

Description	LINFlexD_0	LINFlexD_1	LINFlexD_2
Number of filters (no_of_filters)	0	0	0
Number of Tx DMA channels (2TX_CH_NUM)	1	1	1
Number of Rx DMA channels (2RX_CH_NUM)	1	1	1
Slave mode enabled	1	1	1
UART mode enabled	1	1	1
Auto-synchronization enabled	0	0	0
DMA support	Yes	Yes	Yes

Figure 3.2 LINFlexD configuration in S32G2 Reference manual

LINFLEXD_0 channel can be found in the *S32G2_IOMUX.xlsx* or *S32R45_IOMUX.xlsx* or *S32G3_IOMUX.xlsx* file from Reference manual with its name is LINFlex_0. The Pin-Muxing for S32G2 is:

PK_15	175 SIUL_OFFCC	0000_0000	0x440104FC	GPIO[175]	SIUL_OFFCC	General Purpose I/O 175	I/O
PK_15		0000_0001		LLCE_LIN0_TX	LLCE_LIN_0	LLCE LIN Transmit Data	O
PK_15		0000_0010		LIN0_TX	LINFlex_0	LIN Transmit Data	O
PK_15		0000_0011		LLCE_LPSPiO_PCS3_O	LLCE_LPSPiO_0	LLCE LPSPiO Chip Select 3	O
PK_15		0000_0100		DSPiS_SCK_O	DSPi_5	SPI Clock	O
PK_15		0000_0101		I2C1_SCL_O	I2C_1	I2C Bidirectional Serial Clock	O
PK_15		0000_0110		LLCE_LPSPiS_SIN_O	LLCE_LPSPi_3	LLCE LPSPi Serial Data In	O
PK_15	1006 SIUL_OFFCC	0000_0100	0x440111F8	DSPiS_SCK_I	DSPi_5	SPI Clock	I
PK_15	717 SIUL_OFFCC	0000_0110	0x44010D74	I2C1_SCL_I	I2C_1	I2C Bidirectional Serial Clock	I
PK_15	798 SIUL_OFFCC	0000_0011	0x44010E88	LLCE_LPSPiO_PCS3_I	LLCE_LPSPi_0	LLCE LPSPi Chip Select 3	I
PL_00	176 SIUL_OFFCC	0000_0000	0x44010500	GPIO[176]	SIUL_OFFCC	General Purpose Input 176	I
PL_00	790 SIUL_OFFCC	0000_0010	0x44010E98	LLCE_LIN0_RX	LLCE_LIN_0	LLCE LIN Receive Data	I
PL_00				WKUP16	WKPU	Wakeup Input 16	I
PL_00	512 SIUL_CC	0000_0100	0x4009CA40	LIN0_RX	LINFlex_0	LIN Receive Data	I

Figure 3.3 Pin-Muxing of LINFlex_0 on S32G2 platform

For transmitting data signal, TXD should be connected to pin PK[15]. And for receiving data signal, RXD should be connected to pin PL[00].

The Pin-Muxing for S32G3 is:

Port	Pair Port	CR	CR Instance Name	SSS	Addr	Function	Module	Description	D
PC_09				0000_0001		LIN0_TX	LINFlex_0	LIN Transmit Data	O
PC_10		512	SIUL_CC	0000_0010		LIN0_RX	LINFlex_0	LIN Receive Data	I

Figure 3.4 Pin-Muxing of LINFlex_0 for S32G3 platform

For transmitting data signal, TXD should be connected to pin PC[9]. And for receiving data signal, RXD should be connected to pin PC[10].

The Pin-Muxing for S32R45 is:

PC_09	41	SIUL_CC	0000_0000	0x4009_C2E4	GPIO[41]	SIUL_CC	General Purpose I/O	I/O
			0000_0001		LIN0_TX	LINFlex_0	LIN Transmit Data	O
			0000_0010		CAN3_TX	FlexCAN_3	CAN Transmit Channel	O
			0000_0011		FR_DBG[0]	FlexRay_0		O
			0000_0100		DSPi4_SCK_O	DSPi_4	Deserial Serial Peripheral Interface	O
PC_10	647	SIUL_OFFCC	0000_0010	0x4403_CC5C	DSPi4_SCK_I	DSPi_4	Deserial Serial Peripheral Interface	I
	42	SIUL_CC	0000_0000	0x4009_C2E8	GPIO[42]	SIUL_CC	General Purpose I/O	I/O
			0000_0001		CAN1_TX	FlexCAN_1	CAN Transmit Channel	O
			0000_0010		DSPi1_SCK_O	DSPi_1	Deserial Serial Peripheral Interface	O
			0000_0011		DSPi4_SOUT_O	DSPi_4	Deserial Serial Peripheral Interface	O
PC_10			0000_0100		FR_DBG[1]	FlexRay_0		O
	512	SIUL_CC	0000_0010	0x4009_CA40	LIN0_RX	LINFlex_0	LIN Receive Data	I
	622	SIUL_OFFCC	0000_0011	0x4403_CBF8	DSPi1_SCK_I	DSPi_1	Deserial Serial Peripheral Interface	I
	643	SIUL_OFFCC	0000_0010	0x4403_CC64	DSPi4_SIN_I	DSPi_4	Deserial Serial Peripheral Interface	I

Figure 3.5 Pin-Muxing of LINFlex_0 for S32R45 platform

For transmitting data signal, TXD should be connected to pin PC[9]. And for receiving data signal, RXD should be connected to pin PC[10].

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR LIN Driver software specification in some places. The table below identifies the AUTOSAR requirements that are not implemented or out of scope for the LIN Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently or out of scope for the LIN driver.

Requirement	Status	Description	Notes
SWS_Lin_00177	N/S	In case several LIN driver instances (of same or different vendor) are implemented in one ECU the file names, API names, and published parameters must be modified such that no two definitions with the same name are generated. The name shall be extended according to SRS_BSW_00347 with a Vendor Id (needed to distinguish LIN drivers from different vendors) and a Vendor specific name (needed to distinguish different hardware units implemented by one Vendor).	Not apply for S32XX.
SWS_Lin_00055	N/S	The Lin module shall fulfill all design and implementation guidelines as described in Specification of C Implementation Rules AUTOSAR_TR_C - ImplementationRules.pdf.	Requirement already covered by process.
SWS_Lin_00026	N/S	If the LIN hardware unit cannot queue the bytes for transmission or reception (e.g. simple UART implementation), the LIN driver shall provide a temporary communication buffer.	The LIN hardware already has a data buffer built-in.
SWS_Lin_00039	N/S	Values that can be configured are hardware dependent. Therefore, the rules and constraints cannot be given in the standard.	This is not a requirement.
SWS_Lin_00999	N/S	These requirements are not applicable to this specification.	This is not a requirement.

Requirement	Status	Description	Notes
SWS_Lin_00201	N/S	For different LIN hardware units a separate LIN driver needs to be implemented. It is up to the implementer to adapt the driver to the different instances of similar LIN channels.	Rejection reason: LIN driver is designed to adapt to different LIN hardware units since.
SWS_Lin_00099	N/S	If development error detection for the Lin module is enabled: the function Lin_Init shall check the parameter Config for being within the allowed range. If Config is not in the allowed range, the function Lin_Init shall raise the development error LIN_E_INVA - LID_POINTER.	Requirement is marked Rejected, as it is replaced by CPR_RTD_00255

3.5 Driver Limitations

In Ip layer configuration, number of stop bits and checksum control are not allowed to be configurable by user.

3.6 Driver usage and configuration tips

3.6.1 Frame Timeout Disable Feature

The 'Lin Frame Timeout Disable' checkbox in AutosarExt container should be enabled in order to have this feature active. If LinFrameTimeoutDisable is ON then Lin driver will accept the frame that is longer than Maximal Frame Length. For the LIN driver, timeout feature is applied to both LIN 2.1 master and slave nodes, in response frame reception.

Note

In slave mode, this feature must be set to false to avoid errata e7274

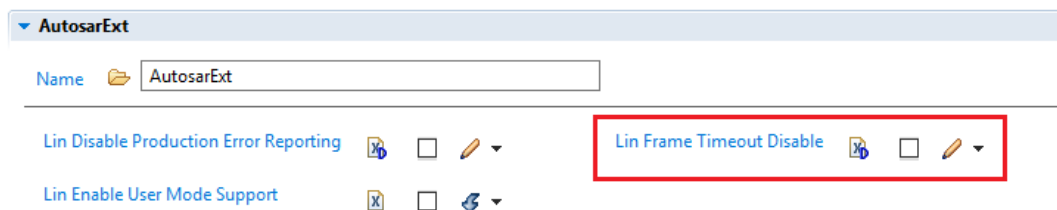


Figure 3.6 Frame Timeout Disable Feature configuration node in EB Tresos.

3.6.2 Dual Clock Feature

Lin driver allow dynamic change of working frequency. The LinClockRef_Alternate node is from a link: Lin/↔ LinGlobalConfig/[Configuration_name]/LinChannel/General should be enabled in order to have this feature active. And the frequency can be changed if Lin_SetClockMode function is call, after calling Lin_Init function. Our recommended usage for this API is to call it when the driver is in a lower power state but still in active use.

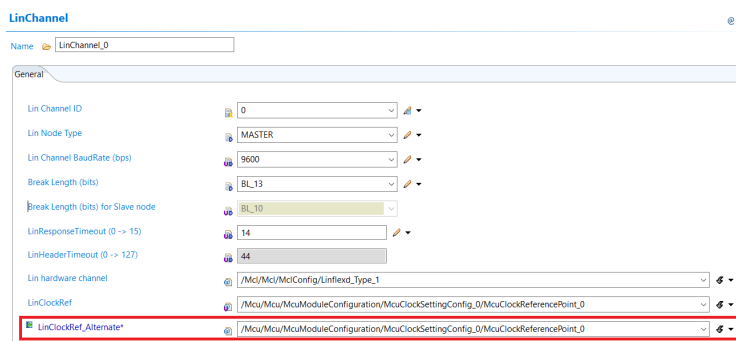


Figure 3.7 Dual Clock Feature configuration node in EB Tresos.

3.6.3 How to configure multicore

Set multicore support node of Lin module is ON to partitions can be referred.

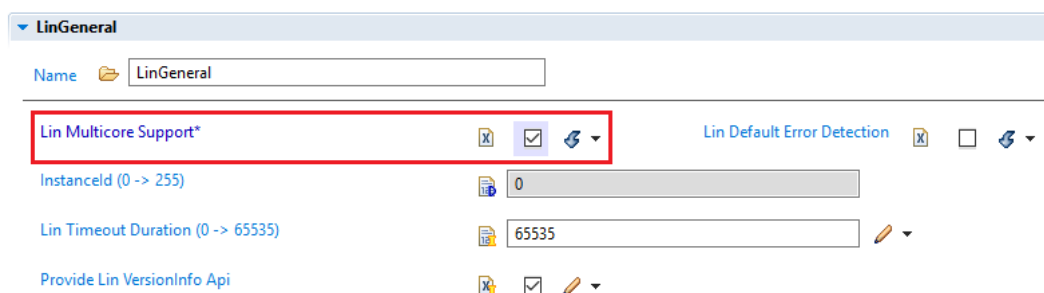


Figure 3.8 Enable Multicore support for Lin module.

A partition will be assigned for each Lin channel. Each LinChannel selects only a partition (reference from ECUC).

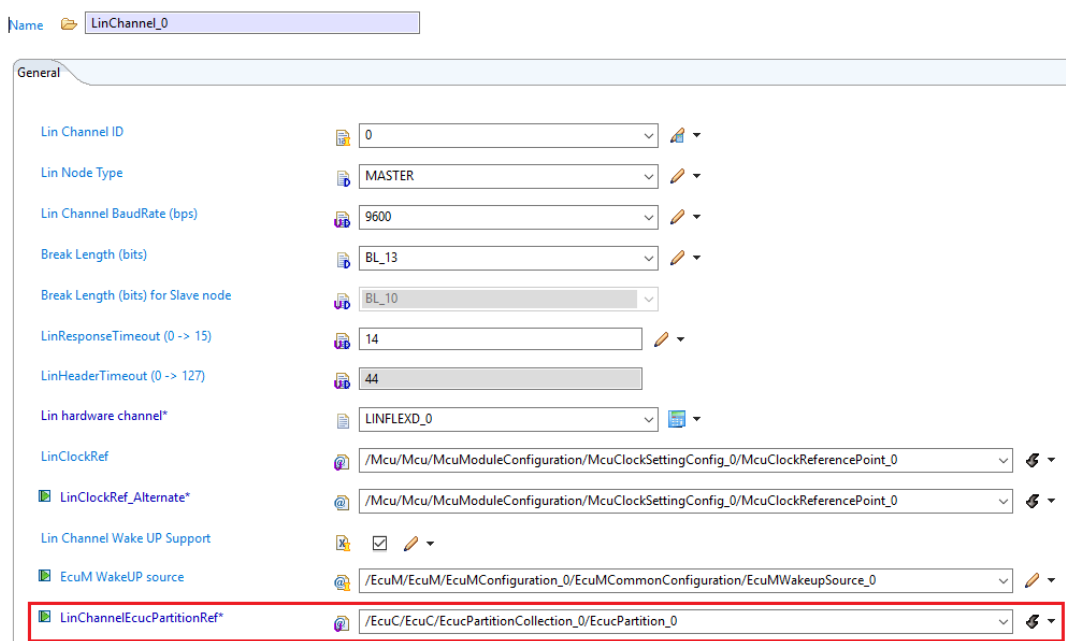


Figure 3.9 Select partition for Lin channel.

Each partition will be referred to a core by Os. Each OsApplication will be set to one partition maps to one core.

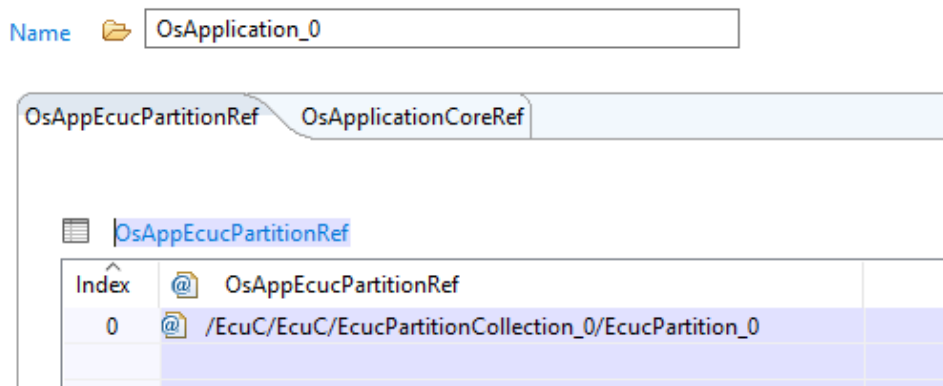


Figure 3.10 Partition referred in Os.

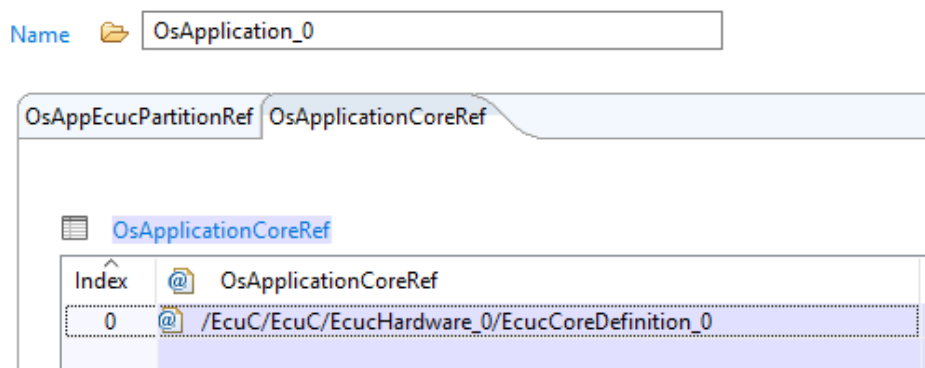


Figure 3.11 Core referred in Os.

In EcuC configuration, each EcucCoreDefinition will selected a Core.

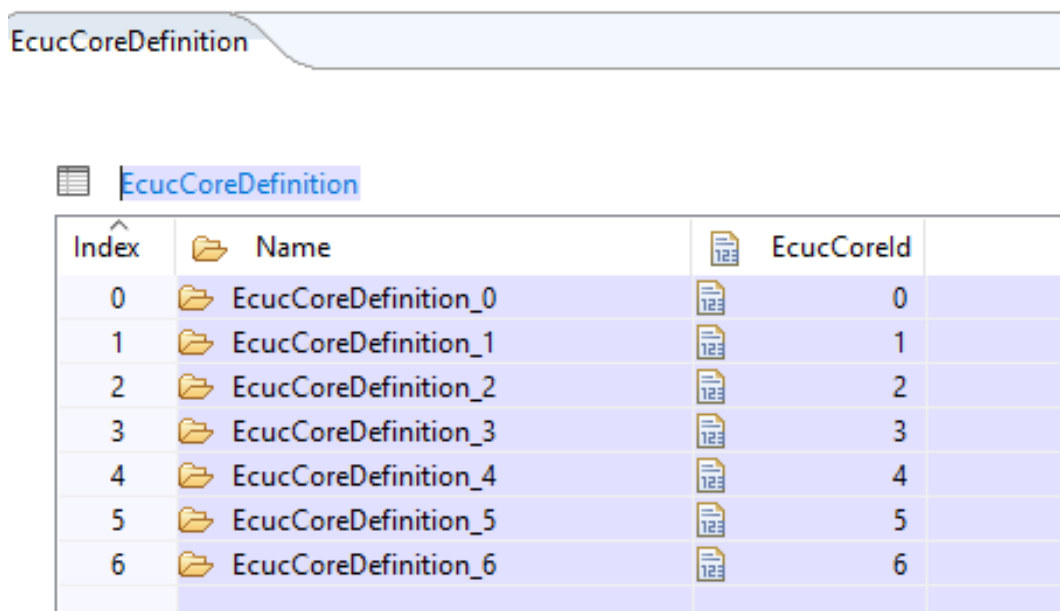


Figure 3.12 EcucCoreDefinition selected a core.

3.7 Runtime errors

The Lin driver generates the following DEM errors at runtime.

Function	Error Code	Condition triggering the error
Lin_GoToSleep()	Lin_E_TIMEOUT	Timeout caused by hardware error waiting for cancellation of current byte. No sleep command will be sent , and Lin driver will not enter sleep state.

Function	Error Code	Condition triggering the error
Lin_GoToSleepInternal()	Lin_E_TIMEOUT	Timeout caused by hardware error waiting for cancellation of current byte. Lin driver will not enter sleep state.
Lin_SendFrame()	Lin_E_TIMEOUT	Timeout caused by hardware error waiting for cancellation of current byte. New frame is not sent.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Lin](#)
 - Container [AutosarExt](#)
 - * Parameter [LinDisableDemReportErrorStatus](#)
 - * Parameter [LinFrameTimeoutDisable](#)
 - * Parameter [LinEnableUserModeSupport](#)
 - Container [LinGeneral](#)
 - * Parameter [LinMulticoreSupport](#)
 - * Parameter [LinDevErrorDetect](#)
 - * Parameter [LinIndex](#)
 - * Parameter [LinTimeoutMethod](#)
 - * Parameter [LinTimeoutDuration](#)
 - * Parameter [LinVersionInfoApi](#)
 - * Reference [LinEcucPartitionRef](#)
 - Container [LinDemEventParameterRefs](#)
 - * Reference [LIN_E_TIMEOUT](#)
 - Container [LinGlobalConfig](#)
 - * Container [LinChannel](#)
 - Parameter [LinChannelId](#)
 - Parameter [LinNodeType](#)
 - Parameter [LinChannelBaudRate](#)
 - Parameter [BreakLength](#)
 - Parameter [SlaveSyncBreakLength](#)
 - Parameter [LinResponseTimeout](#)
 - Parameter [LinHeaderTimeout](#)
 - Parameter [LinHwChannel](#)
 - Parameter [LinChannelWakeupSupport](#)
 - Reference [LinClockRef](#)
 - Reference [LinClockRef_Alternate](#)
 - Reference [LinChannelEcuMWakeupSource](#)

- Reference [LinChannelEcucPartitionRef](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module Lin

Configuration of the Lin (Local Interconnect Network) module.

Included containers:

- [AutosarExt](#)
- [LinGeneral](#)
- [LinDemEventParameterRefs](#)
- [LinGlobalConfig](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container AutosarExt

AutosarExt

Autosar Requirements:

This container contains the global configuration parameters of the Non-Autosar Lin driver.

This container is a MultipleConfigurationContainer, i.e. this container and

its sub-containers exist once per configuration set.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter LinDisableDemReportErrorStatus

LinDisableDemReportErrorStatus

Switches the Diagnostic Error Reporting and Notification OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.4 Parameter LinFrameTimeoutDisable

LinFrameTimeoutDisable

The master will accept the frame that is longer than TFrame__Maximum.

To avoid Errata ERR007274 this feature will be only applied for Master mode, Timeout will be always used by Slave mode and not depend on this node.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.5 Parameter LinEnableUserModeSupport

When this parameter is enabled, the MDL module will adapt to run from User Mode, with the following measures:

- a) configuring REG_PROT for ABC1, ABC2 IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1
- b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.
- c) other module specific measures

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.6 Container LinGeneral

LinGeneral

Autosar Requirements: ECUC_Lin_00183

This container contains the parameters related to each LIN Driver Unit.

This container is a MultipleConfigurationContainer, i.e. this container and

its sub-containers exist once per configuration set.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.7 Parameter LinMulticoreSupport

This parameter determine multi-core feature will be used in Lin driver.

If LinMulticoreSupport is disabled, then for all the variants no partition shall be defined.

If LinMulticoreSupport is enabled, at least one EcucPartition needs to be defined (in all variants).

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.8 Parameter LinDevErrorDetect

LinDevErrorDetect

Autosar Requirements: ECUC_Lin_00066

Switches the Default Error Detection and Notification ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.9 Parameter LinIndex

Autosar Requirements: ECUC_Lin_00179

Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.

Note, this parameter is not used in the current implementation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.10 Parameter LinTimeoutMethod

Vendor specific:

LinTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

This parameter is used to select between different OsIf counter implementations. For additional details, please refer to the OsIf documentation.

Note: If SystemTimer or CustomTimer are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.11 Parameter LinTimeoutDuration

Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops. If LinTimeoutMethod is OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM, LinTimeoutDuration is in microsecond value. If LinTimeoutMethod is OSIF_COUNTER_DUMMY, the LinTimeoutDuration is number of wait loop.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	65535
min	0

4.12 Parameter LinVersionInfoApi

LinVersionInfoApi

Autosar Requirements: ECUC_Lin_00067

Switches the Lin_GetVersionInfo function ON or OFF.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.13 Reference LinEcucPartitionRef

Maps the Lin driver to zero or multiple ECUC partitions to make the modules API available in this partition.

The Lin driver will operate as an independent instance in each of the partitions.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC

Property	Value
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.14 Container LinDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.15 Reference LIN_E_TIMEOUT

Reference to the DemEventParameter which shall be issued when the error "Timeout caused by hardware error" has occurred.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.16 Container LinGlobalConfig

This container contains the global configuration parameter of the Lin driver. This container is a `MultipleConfigurationContainer`, i.e. this container and its sub-containers exit once per configuration set.

Included subcontainers:

- LinChannel

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.17 Container LinChannel

This container contains the configuration (parameters) of the LIN Controller(s).

Note: "User should use unique names for naming the LIN channels across different LinGlobalConfig Sets."

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	\$32-BIN-DEF \$32-BIN-DEF

4.18 Parameter LinChannelId

Identifies the LIN channel. Replaces LIN_CHANNEL_INDEX_NAME from the LIN SWS.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	0

4.19 Parameter LinNodeType

LinNodeType

Autosar Requirements: ECUC_Lin_00191

Specifies the LIN node type of this channel: Master or Slave node

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	MASTER
literals	['MASTER', 'SLAVE']

4.20 Parameter LinChannelBaudRate

LinChannelBaudRate

Autosar Requirements: ECUC_Lin_00180

Specifies the baud rate of the LIN channel in 'bps'. Valid range: 4800..20000.

Note: The minimum value of the baudrate is 4800bps (AUTOSAR Standard: 1000bps), this information was specified in data sheet

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	9600
max	20000
min	1000

4.21 Parameter BreakLength

Vendor specific: Defines the break length in bits.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	BL_13
literals	['BL_13', 'BL_14', 'BL_15', 'BL_16', 'BL_17', 'BL_18', 'BL_19', 'BL_20', 'BL_21', 'BL_22', 'BL_23', 'BL_36', 'BL_50']

4.22 Parameter SlaveSyncBreakLength

Vendor specific: Defines the break length in bits which can detect by Slave node.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	BL_10
literals	['BL_10', 'BL_11']

4.23 Parameter LinResponseTimeout

Vendor specific:

Response timeout value.

LINTOCR[RTO] will be configured by this node.

This is the response timeout duration (in bit time) for 1 byte.

The reset value is 0Eh = 14, corresponding to $T_Response_Maximum = 1.4 \times T_Response_Nominal$. Here, 1.4 corresponds to $(LINTOCR[RTO])/10$.

This node is only configured if AutosarExt/LinFrameTimeoutDisable is false.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	14
max	15
min	0

4.24 Parameter LinHeaderTimeout

Vendor specific:

Header timeout value

LINTOCR[HTO] will be configured by this node.

This field contains the header timeout duration (in bit time).

For slave, these bits should be programmed without considering 11 bits of break.

Please see sub-chapter "Timeout error" in Reference Manual to know how to calculate this value.

This node is only configured if AutosarExt/LinFrameTimeoutDisable is false and LinNodeType is SLAVE.

Note:

This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	14
max	127
min	0

4.25 Parameter LinHwChannel

Selects the Lin Hardware Channel.

Note:

Vendor Specific: This Parameter is an Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: PRE-COMPILE
default Value	LINFLEXD_0
literals	['LINFLEXD_0', 'LINFLEXD_1', 'LINFLEXD_2']

4.26 Parameter LinChannelWakeupSupport

LinChannelWakeupSupport

Autosar Requirements: LIN182_Conf

Specifies if the LIN hardware channel supports wake up functionality.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.27 Reference LinClockRef

Reference to the LIN clock source configuration, which is set in the MCU driver configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.28 Reference LinClockRef_Alternate

Vendor specific: Alternate reference to the LIN clock source configuration, which is set in the MCU driver configuration, used in Low Power Mode.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.29 Reference LinChannelEcuMWakeupSource

This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuM/EcuMConfiguration/EcuMCommon↔ Configuration/EcuMWakeupSource

4.30 Reference LinChannelEcucPartitionRef

Maps one single Lin channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions where the Lin driver is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.31 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.32 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.33 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.34 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.35 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	82

Property	Value
max	82
min	82

4.36 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.37 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.38 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.39 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.40 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

LIN Driver	38
LIN	48
LIN_IPW	49
LINFLEXD_IP	50

Chapter 6

Module Documentation

6.1 LIN Driver

6.1.1 Detailed Description

Data Structures

- struct [Lin_ChannelConfigType](#)
LIN channel configuration type structure. [More...](#)
- struct [Lin_ConfigType](#)
LIN driver configuration type structure. [More...](#)

Macros

- `#define LIN_UNINIT`
LIN driver states.
- `#define LIN_INIT`
LIN driver states.
- `#define LIN_CH_SLEEP_PENDING`
LIN Channel states.
- `#define LIN_CH_SLEEP_STATE`
LIN Channel states.
- `#define LIN_CH_OPERATIONAL`
LIN Channel states.
- `#define LIN_GETSTATUS_ID`
API service ID for Lin_GetStatus() function.
- `#define LIN_GETVERSIONINFO_ID`
API service ID for Lin_GetVersionInfo() function.
- `#define LIN_GOTOSLEEP_ID`
API service ID for Lin_GoToSleep() function.
- `#define LIN_GOTOSLEEPINTERNAL_ID`

- *API service ID for `Lin_GoToSleepInternal()` function.*
- `#define LIN_INIT_ID`
- *API service ID for `Lin_Init()` function.*
- `#define LIN_SENDFRAME_ID`
- *API service ID for `Lin_SendFrame()` function.*
- `#define LIN_WAKEUP_ID`
- *API service ID for `Lin_WakeUp()` function.*
- `#define LIN_WAKEUPINTERNAL_ID`
- *API service ID for `Lin_WakeupInternal()` function.*
- `#define LIN_CHECKWAKEUP_ID`
- *API service ID for `Lin_CheckWakeup()` function.*
- `#define LIN_TIMEOUT_ERROR`
- *Return code for timeout error.*

Function Reference

- Std_ReturnType `Lin_CheckWakeup` (uint8 Channel)
Validates for upper layers the wake up of LIN channel.
- void `Lin_Init` (const `Lin_ConfigType` *Config)
Initializes the LIN module.
- Std_ReturnType `Lin_GoToSleepInternal` (uint8 Channel)
Same function as `Lin_Ipw_GoToSleep()` but without sending a go-to-sleep-command on the bus.
- Std_ReturnType `Lin_Wakeup` (uint8 Channel)
Generates a wake up pulse.
- Std_ReturnType `Lin_WakeupInternal` (uint8 Channel)
Wake up the LIN channel.

6.1.2 Data Structure Documentation

6.1.2.1 struct `Lin_ChannelConfigType`

LIN channel configuration type structure.

This is the type of the external data structure containing the overall initialization data for one LIN Channel. A pointer to such a structure is provided to the LIN channel initialization routine for configuration of the LIN hardware channel.

Definition at line 143 of file `Lin_Types.h`.

Data Fields

Type	Name	Description
uint8	LinChannelID	
const <code>Lin_HwConfigType</code> *	ChannelConfigPtr	Lin Channel ID. !<
uint32	ChannelCoreId	LIN Hardware configuration pointer. !<
boolean	AllocatedPartition	LIN Channel core id. !<

6.1.2.2 struct Lin_ConfigType

LIN driver configuration type structure.

This is the type of the pointer to the external data LIN Channels. A pointer of such a structure is provided to the LIN driver initialization routine for configuration of the LIN hardware channel.

Definition at line 163 of file Lin_Types.h.

Data Fields

- const [Lin_ChannelConfigType](#) * [Lin_ChannelPtr](#) [LIN_HW_MAX_MODULES]
Partition core id is assigned for this configuration.

6.1.2.2.1 Field Documentation

6.1.2.2.1.1 Lin_ChannelPtr const [Lin_ChannelConfigType](#)* [Lin_ChannelPtr](#) [LIN_HW_MAX_MODULES]

Partition core id is assigned for this configuration.

!<

Hardware channel.

Constant pointer of the constant external data structure containing the overall initialization data for all the configured LIN Channels.

Definition at line 172 of file Lin_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 LIN_UNINIT

```
#define LIN_UNINIT
```

LIN driver states.

The state LIN_UNINIT means that the Lin module has not been initialized yet and cannot be used.

Definition at line 228 of file Lin.h.

6.1.3.2 LIN_INIT

```
#define LIN_INIT
```

LIN driver states.

The LIN_INIT state indicates that the LIN driver has been initialized, making each available channel ready for service.

Definition at line 237 of file Lin.h.

6.1.3.3 LIN_CH_SLEEP_PENDING

```
#define LIN_CH_SLEEP_PENDING
```

LIN Channel states.

go-to-sleep-command has been issued on the bus, LIN channel stay at this state until Lin_GetStatus() is called

Definition at line 246 of file Lin.h.

6.1.3.4 LIN_CH_SLEEP_STATE

```
#define LIN_CH_SLEEP_STATE
```

LIN Channel states.

The detection of a wake-up pulse is enabled. The LIN hardware is into a low power mode if such a mode is provided by the hardware.

Definition at line 256 of file Lin.h.

6.1.3.5 LIN_CH_OPERATIONAL

```
#define LIN_CH_OPERATIONAL
```

LIN Channel states.

The individual channel has been initialized (using at least one statically configured data set) and is able to participate in the LIN cluster.

Definition at line 266 of file Lin.h.

6.1.3.6 LIN_GETSTATUS_ID

```
#define LIN_GETSTATUS_ID
```

API service ID for `Lin_GetStatus()` function.

Parameters used when raising an error or exception.

Definition at line 276 of file `Lin.h`.

6.1.3.7 LIN_GETVERSIONINFO_ID

```
#define LIN_GETVERSIONINFO_ID
```

API service ID for `Lin_GetVersionInfo()` function.

Parameters used when raising an error or exception.

Definition at line 284 of file `Lin.h`.

6.1.3.8 LIN_GOTOSLEEP_ID

```
#define LIN_GOTOSLEEP_ID
```

API service ID for `Lin_GoToSleep()` function.

Parameters used when raising an error or exception.

Definition at line 292 of file `Lin.h`.

6.1.3.9 LIN_GOTOSLEEPINTERNAL_ID

```
#define LIN_GOTOSLEEPINTERNAL_ID
```

API service ID for [Lin_GoToSleepInternal\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 300 of file `Lin.h`.

6.1.3.10 LIN_INIT_ID

```
#define LIN_INIT_ID
```

API service ID for [Lin_Init\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 308 of file Lin.h.

6.1.3.11 LIN_SENDFRAME_ID

```
#define LIN_SENDFRAME_ID
```

API service ID for [Lin_SendFrame\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 316 of file Lin.h.

6.1.3.12 LIN_WAKEUP_ID

```
#define LIN_WAKEUP_ID
```

API service ID for [Lin_WakeUp\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 324 of file Lin.h.

6.1.3.13 LIN_WAKEUPINTERNAL_ID

```
#define LIN_WAKEUPINTERNAL_ID
```

API service ID for [Lin_WakeupInternal\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 332 of file Lin.h.

6.1.3.14 LIN_CHECKWAKEUP_ID

#define LIN_CHECKWAKEUP_ID

API service ID for [Lin_CheckWakeup\(\)](#) function.

Parameters used when raising an error or exception.

Definition at line 340 of file Lin.h.

6.1.3.15 LIN_TIMEOUT_ERROR

#define LIN_TIMEOUT_ERROR

Return code for timeout error.

Definition at line 119 of file Lin_Types.h.

6.1.4 Function Reference

6.1.4.1 Lin_CheckWakeup()

```
Std_ReturnType Lin_CheckWakeup (
    uint8 Channel )
```

Validates for upper layers the wake up of LIN channel.

This function identifies if the addressed LIN channel has been woken up by the LIN bus transceiver. It checks the wake up flag from the addressed LIN channel which must be in sleep mode and have the wake up signal.

Parameters

in	<i>Channel</i>	LIN channel to be addressed.
----	----------------	------------------------------

Return values

<i>E_NOT_OK</i>	If the LIN Channel is not valid or LIN driver is not initialized or the addressed LIN Channel is not in sleep state.
<i>E_OK</i>	Otherwise.

6.1.4.2 Lin_Init()

```
void Lin_Init (
    const Lin_ConfigType * Config )
```

Initializes the LIN module.

This function performs software initialization of LIN driver:

- Clears the shadow buffer of all available Lin channels
- Sets all the available channels to sleep mode and configures their state machine to LIN_CH_SLEEP_STATE.
- Set driver state machine to LIN_INIT.

Parameters

in	<i>Config</i>	- Pointer to LIN driver configuration set.
----	---------------	--

Returns

void

Precondition

-

6.1.4.3 Lin_GoToSleepInternal()

```
Std_ReturnType Lin_GoToSleepInternal (
    uint8 Channel )
```

Same function as Lin_Ipw_GoToSleep() but without sending a go-to-sleep-command on the bus.

This function stops any ongoing transmission and put the Channel in sleep mode (then LIN hardware enters a reduced power operation mode).

Parameters

in	<i>Channel</i>	LIN channel to be addressed.
----	----------------	------------------------------

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	In case of a timeout situation only.
<i>E_OK</i>	Otherwise.

Precondition

: `Lin_Init` function must be called before this API.

6.1.4.4 `Lin_Wakeup()`

```
Std_ReturnType Lin_Wakeup (  
    uint8 Channel )
```

Generates a wake up pulse.

This function shall sent a wake up signal to the LIN bus and put the LIN channel in `LIN_CH_OPERATIONAL` state.

Parameters

in	<i>Channel</i>	LIN channel to be addressed.
----	----------------	------------------------------

Returns

`Std_ReturnType`.

Return values

<i>E_NOT_OK</i>	If the LIN driver is not in sleep state or LIN Channel is not valid or LIN driver is not initialized.
<i>E_OK</i>	Otherwise.

Precondition

: `Lin_Init` function must be called before this API.

6.1.4.5 `Lin_WakeupInternal()`

```
Std_ReturnType Lin_WakeupInternal (  
    uint8 Channel )
```

Wake up the LIN channel.

This function shall put the LIN channel in LIN_CH_OPERATIONAL state without sending a wake up signal to the LIN bus

Parameters

in	<i>Channel</i>	LIN channel to be addressed.
----	----------------	------------------------------

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the LIN driver is not in sleep state or LIN Channel is not valid or LIN driver is not initialized.
<i>E_OK</i>	Otherwise.

Precondition

: Lin_Init function must be called before this API.

6.2 LIN

6.2.1 Detailed Description

6.3 LIN_IPW

6.3.1 Detailed Description

Enum Reference

- enum [Lin_Ipw_NodeType](#)

Definition of the LIN node type of the current channel.

6.3.2 Enum Reference

6.3.2.1 Lin_Ipw_NodeType

enum [Lin_Ipw_NodeType](#)

Definition of the LIN node type of the current channel.

This type enumerates the node types for a LIN channel.

Enumerator

LIN_MASTER_NODE	This is used for Master node.
LIN_SLAVE_NODE	This is used for Slave node.

Definition at line 127 of file Lin_Ipw_Types.h.

6.4 LINFLEXD_IP

6.4.1 Detailed Description

Data Structures

- struct [Linflexd_Lin_Ip_PduType](#)
Define type for an structure containing information regarding the LIN Frame . [More...](#)
- struct [Linflexd_Lin_Ip_StateStructType](#)
Runtime state of the LIN driver. [More...](#)

Macros

- `#define LINFLEXD_LIN_IP_TX_NO_COMMAND`
- `#define LINFLEXD_LIN_IP_TX_MASTER_RES_COMMAND`
- `#define LINFLEXD_LIN_IP_TX_SLAVE_RES_COMMAND`
- `#define LINFLEXD_LIN_IP_TX_SLAVE_TO_SLAVE_COMMAND`
- `#define LINFLEXD_LIN_IP_LINS_SLEEP_MODE_U8`
- `#define LINFLEXD_LIN_IP_LINS_INIT_MODE_U8`
- `#define LINFLEXD_LIN_IP_LINS_IDLE_MODE_U8`
- `#define LINFLEXD_LIN_IP_LINS_SYNC_BREAK_U8`
- `#define LINFLEXD_LIN_IP_LINS_SYNC_DEL_U8`
- `#define LINFLEXD_LIN_IP_LINS_SYNC_FIELD_U8`
- `#define LINFLEXD_LIN_IP_LINS_IDENTIFIER_FIELD_U8`
- `#define LINFLEXD_LIN_IP_LINS_HEADER_RECEPT_TRANS_U8`
- `#define LINFLEXD_LIN_IP_LINS_DATA_RECEPT_TRANS_U8`
- `#define LINFLEXD_LIN_IP_LINS_CHECKSUM_U8`

Types Reference

- typedef void(* [Linflexd_Lin_Ip_CallbackType](#)) (const uint8 Instance, const [Linflexd_Lin_Ip_StateStructType](#) *LinState)
LIN Driver callback function type.

Enum Reference

- enum [Linflexd_Lin_Ip_EventIdType](#)
Defines types for an enumerating event related to an Identifier.
- enum [Linflexd_Lin_Ip_NodeStateType](#)
Define type for an enumerating LIN Node state.
- enum [Linflexd_Lin_Ip_TransferStatusType](#)
This type defines a range of transfer status.
- enum [Linflexd_Lin_Ip_StatusType](#)
This type defines a range of specific status for return operation.
- enum [Linflexd_Lin_Ip_FrameCsModelType](#)
- enum [Linflexd_Lin_Ip_FrameResponseType](#)

Function Reference

- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_Init](#) (const uint8 Instance, const Linflexd_Lin_Ip_User↔ConfigType *LinUserConfig)
Initializes an LIN_LINFLEXD instance for LIN Network.
- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_Deinit](#) (const uint8 Instance)
Shuts down the LIN_LINFLEXD by disabling interrupts and transmitter/receiver.
- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_SendFrame](#) (const uint8 Instance, const [Linflexd_Lin_Ip_PduType](#) *PduInfoPtr)
Sends frame out through the LIN_LINFLEXD module using non-blocking method. This enables an a-sync method for transmitting data. Non-blocking means that the function returns immediately. The application has to get the transmit status to know when the transmit is complete. This function will calculate the checksum byte and send it with the frame data after header transmitted. If frame size is equal to 0 or greater than 8 or node's current state is in SLEEP mode then the function will return STATUS_ERROR. If isBusBusy is currently true then the function will return STATUS_BUSY.
- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_AbortTransferData](#) (const uint8 Instance)
Aborts an on-going non-blocking transmission/reception. While performing a non-blocking transferring data, users can call this function to terminate immediately the transferring.
- [Linflexd_Lin_Ip_NodeStateType Linflexd_Lin_Ip_GetCurrentNodeState](#) (const uint8 Instance)
This function is used to retrieve the current state of the lin node.
- [Linflexd_Lin_Ip_TransferStatusType Linflexd_Lin_Ip_GetStatus](#) (const uint8 Instance, uint8 **Buffer)
This function returns the status of the current non-blocking transfer. If a response reception has been successfully received, Buffer will be referenced to receive buffer.
- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_GoToSleepMode](#) (const uint8 Instance)
This function puts current node to sleep mode This function changes current node state to LIN_NODE_STATE↔_SLEEP_MODE.
- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_GotoIdleState](#) (const uint8 Instance)
This function puts current node to idle mode.
- [Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_SendWakeupSignal](#) (const uint8 Instance)
Sends a wakeup signal through the LIN_LINFLEXD interface.

6.4.2 Data Structure Documentation

6.4.2.1 struct Linflexd_Lin_Ip_PduType

Define type for an structure containing information regarding the LIN Frame .

Definition at line 201 of file Linflexd_Lin_Ip_Types.h.

Data Fields

- uint8 [Pid](#)
LIN frame identifier.
- [Linflexd_Lin_Ip_FrameCsModelType Cs](#)
Checksum model type.
- [Linflexd_Lin_Ip_FrameResponseType Drc](#)
Response type.
- uint8 [Dl](#)
Data length.
- uint8 * [SduPtr](#)
Pointer to Sdu.

6.4.2.1.1 Field Documentation

6.4.2.1.1.1 **Pid** `uint8 Pid`

LIN frame identifier.

Definition at line 203 of file `Linflexd_Lin_Ip_Types.h`.

6.4.2.1.1.2 **Cs** `Linflexd_Lin_Ip_FrameCsModelType Cs`

Checksum model type.

Definition at line 204 of file `Linflexd_Lin_Ip_Types.h`.

6.4.2.1.1.3 **Drc** `Linflexd_Lin_Ip_FrameResponseType Drc`

Response type.

Definition at line 205 of file `Linflexd_Lin_Ip_Types.h`.

6.4.2.1.1.4 **Dl** `uint8 Dl`

Data length.

Definition at line 206 of file `Linflexd_Lin_Ip_Types.h`.

6.4.2.1.1.5 **SduPtr** `uint8* SduPtr`

Pointer to Sdu.

Definition at line 207 of file `Linflexd_Lin_Ip_Types.h`.

6.4.2.2 **struct Linflexd_Lin_Ip_StateStructType**

Runtime state of the LIN driver.

Definition at line 217 of file `Linflexd_Lin_Ip_Types.h`.

Data Fields

Type	Name	Description
volatile boolean	IsBusBusy	True if there are data, frame headers being transferred on bus
volatile Linflexd_Lin_Ip_EventIdType	CurrentEventId	Current ID Event
volatile Linflexd_Lin_Ip_NodeStateType	CurrentNodeState	Current Node state
Linflexd_Lin_Ip_PduType *	SlavePduInfoPtr	pointer to PduInfo for slave node
uint8	FrameCommand	Type of command base on response type when Master sends frame.
volatile uint32	LinState	Store the state of LINSR register for further process.

6.4.3 Macro Definition Documentation

6.4.3.1 LINFLEXD_LIN_IP_TX_NO_COMMAND

```
#define LINFLEXD_LIN_IP_TX_NO_COMMAND
```

There is no command requested after initialized or moved to IDLE from SLEEP

Definition at line 83 of file `Linflexd_Lin_Ip_Types.h`.

6.4.3.2 LINFLEXD_LIN_IP_TX_MASTER_RES_COMMAND

```
#define LINFLEXD_LIN_IP_TX_MASTER_RES_COMMAND
```

Master send header and send response

Definition at line 84 of file `Linflexd_Lin_Ip_Types.h`.

6.4.3.3 LINFLEXD_LIN_IP_TX_SLAVE_RES_COMMAND

```
#define LINFLEXD_LIN_IP_TX_SLAVE_RES_COMMAND
```

Master send header and receive response

Definition at line 85 of file `Linflexd_Lin_Ip_Types.h`.

6.4.3.4 LINFLEXD_LIN_IP_TX_SLAVE_TO_SLAVE_COMMAND

```
#define LINFLEXD_LIN_IP_TX_SLAVE_TO_SLAVE_COMMAND
```

Master send header and ignore response

Definition at line 86 of file Linflexd_Lin_Ip_Types.h.

6.4.3.5 LINFLEXD_LIN_IP_LINS_SLEEP_MODE_U8

```
#define LINFLEXD_LIN_IP_LINS_SLEEP_MODE_U8
```

Sleep mode

Definition at line 89 of file Linflexd_Lin_Ip_Types.h.

6.4.3.6 LINFLEXD_LIN_IP_LINS_INIT_MODE_U8

```
#define LINFLEXD_LIN_IP_LINS_INIT_MODE_U8
```

Initialization mode

Definition at line 90 of file Linflexd_Lin_Ip_Types.h.

6.4.3.7 LINFLEXD_LIN_IP_LINS_IDLE_MODE_U8

```
#define LINFLEXD_LIN_IP_LINS_IDLE_MODE_U8
```

Idle mode

Definition at line 91 of file Linflexd_Lin_Ip_Types.h.

6.4.3.8 LINFLEXD_LIN_IP_LINS_SYNC_BREAK_U8

```
#define LINFLEXD_LIN_IP_LINS_SYNC_BREAK_U8
```

Receiving/transmitting sync break in slave/master respectively

Definition at line 92 of file Linflexd_Lin_Ip_Types.h.

6.4.3.9 LINFLEXD_LIN_IP_LINS_SYNC_DEL_U8

```
#define LINFLEXD_LIN_IP_LINS_SYNC_DEL_U8
```

Detected/Completed sync break in slave/master respectively

Definition at line 93 of file Linflexd_Lin_Ip_Types.h.

6.4.3.10 LINFLEXD_LIN_IP_LINS_SYNC_FIELD_U8

```
#define LINFLEXD_LIN_IP_LINS_SYNC_FIELD_U8
```

Receiving/transmitting sync field in slave/master respectively

Definition at line 94 of file Linflexd_Lin_Ip_Types.h.

6.4.3.11 LINFLEXD_LIN_IP_LINS_IDENTIFIER_FIELD_U8

```
#define LINFLEXD_LIN_IP_LINS_IDENTIFIER_FIELD_U8
```

Receiving/transmitting ID field in slave/master respectively

Definition at line 95 of file Linflexd_Lin_Ip_Types.h.

6.4.3.12 LINFLEXD_LIN_IP_LINS_HEADER_RECEPT_TRANS_U8

```
#define LINFLEXD_LIN_IP_LINS_HEADER_RECEPT_TRANS_U8
```

Valid received/transmitted header in slave/master respectively

Definition at line 96 of file Linflexd_Lin_Ip_Types.h.

6.4.3.13 LINFLEXD_LIN_IP_LINS_DATA_RECEPT_TRANS_U8

```
#define LINFLEXD_LIN_IP_LINS_DATA_RECEPT_TRANS_U8
```

Receiving/transmitting data in receiver/transmitter respectively

Definition at line 97 of file Linflexd_Lin_Ip_Types.h.

6.4.3.14 LINFLEXD_LIN_IP_LINS_CHECKSUM_U8

```
#define LINFLEXD_LIN_IP_LINS_CHECKSUM_U8
```

Checksum transmission/reception ongoing

Definition at line 98 of file Linflexd_Lin_Ip_Types.h.

6.4.4 Types Reference

6.4.4.1 Linflexd_Lin_Ip_CallbackType

```
typedef void(* Linflexd_Lin_Ip_CallbackType) (const uint8 Instance, const Linflexd_Lin_Ip_StateStructType *LinState)
```

LIN Driver callback function type.

Definition at line 231 of file Linflexd_Lin_Ip_Types.h.

6.4.5 Enum Reference

6.4.5.1 Linflexd_Lin_Ip_EventIdType

```
enum Linflexd_Lin_Ip_EventIdType
```

Defines types for an enumerating event related to an Identifier.

Enumerator

LINFLEXD_LIN_IP_NO_EVENT	No event yet
LINFLEXD_LIN_IP_WAKEUP_SIGNAL	Received a wakeup signal
LINFLEXD_LIN_IP_RECV_HEADER_OK	Indicate that header was correctly received
LINFLEXD_LIN_IP_SEND_HEADER_OK	Indicate that header was correctly sent
LINFLEXD_LIN_IP_OVERRUN_ERROR	Overrun Error
LINFLEXD_LIN_IP_FRAME_ERROR	Framing Error
LINFLEXD_LIN_IP_READBACK_ERROR	Readback data is incorrect
LINFLEXD_LIN_IP_CHECKSUM_ERROR	Checksum byte is incorrect
LINFLEXD_LIN_IP_TX_COMPLETED	Sending data completed
LINFLEXD_LIN_IP_RX_COMPLETED	Receiving data completed
LINFLEXD_LIN_IP_TIMEOUT_ERROR	Timeout Error
LINFLEXD_LIN_IP_HEADER_RECEPT_ERROR	Header reception error

Definition at line 107 of file Linflexd_Lin_Ip_Types.h.

6.4.5.2 Linflexd_Lin_Ip_NodeStateType

```
enum Linflexd_Lin_Ip_NodeStateType
```

Define type for an enumerating LIN Node state.

Enumerator

LINFLEXD_LIN_IP_NODE_STATE_UNINIT	Uninitialized state
LINFLEXD_LIN_IP_NODE_STATE_SLEEP_MODE	Sleep mode state
LINFLEXD_LIN_IP_NODE_STATE_IDLE	Idle state
LINFLEXD_LIN_IP_NODE_STATE_SEND_HEADER	Send header state
LINFLEXD_LIN_IP_NODE_STATE_RECV_HEADER	Receive header
LINFLEXD_LIN_IP_NODE_STATE_RECV_DATA	Receive data state
LINFLEXD_LIN_IP_NODE_STATE_RECV_DATA_COMPLETED	Receive data completed state
LINFLEXD_LIN_IP_NODE_STATE_SEND_DATA	Send data state
LINFLEXD_LIN_IP_NODE_STATE_SEND_DATA_COMPLETED	Send data completed state

Definition at line 128 of file Linflexd_Lin_Ip_Types.h.

6.4.5.3 Linflexd_Lin_Ip_TransferStatusType

```
enum Linflexd_Lin_Ip_TransferStatusType
```

This type defines a range of transfer status.

Enumerator

LINFLEXD_LIN_IP_STATUS_FAIL	Command has not been accepted.
LINFLEXD_LIN_IP_STATUS_TX_OK	Master: Header or Response successful transmission. Slave: Response successful transmission.
LINFLEXD_LIN_IP_STATUS_TX_BUSY	Master: Ongoing transmission (Header or Response).
LINFLEXD_LIN_IP_STATUS_TX_HEADER_ERROR	Master: Erroneous header transmission such as: Mismatch between sent and read back data or Physical bus error.
LINFLEXD_LIN_IP_STATUS_TX_ERROR	Master or Slave: Erroneous response transmission such as mismatch between sent and read back data, Physical bus error.
LINFLEXD_LIN_IP_STATUS_RX_OK	Master or Slave: Reception of correct response.
LINFLEXD_LIN_IP_STATUS_RX_BUSY	Master or Slave: Ongoing reception. At least one response byte has been received, but the checksum byte has not been received.

Enumerator

LINFLEXD_LIN_IP_STATUS_RX_ERROR	Master or Slave: Erroneous response reception such as: - Framing error - Overrun error - Checksum error or Short response(timeout occurred).
LINFLEXD_LIN_IP_STATUS_RX_NO_RESP↔ ONSE	Master or Slave: No response byte has been received so far.
LINFLEXD_LIN_IP_STATUS_RX_HEADER_OK	Slave: Reception of correct header.
LINFLEXD_LIN_IP_STATUS_RX_HEADER_↔ BUSY	Ongoing header reception.
LINFLEXD_LIN_IP_STATUS_RX_HEADER_↔ ERROR	Slave: Erroneous header reception such as: Break field error, Sync field error, Identifier parity error, Framing error, timeout occurred or Physical bus error.
LINFLEXD_LIN_IP_STATUS_OPERATIONAL	Normal operation; there is no data has been sent or received after channel initialized or switched to IDLE from SLEEP.
LINFLEXD_LIN_IP_STATUS_SLEEP	Sleep state operation.

Definition at line 145 of file Linflexd_Lin_Ip_Types.h.

6.4.5.4 Linflexd_Lin_Ip_StatusType

```
enum Linflexd_Lin_Ip_StatusType
```

This type defines a range of specific status for return operation.

Enumerator

LINFLEXD_LIN_IP_STATUS_SUCCESS	Successful operation.
LINFLEXD_LIN_IP_STATUS_ERROR	Failed operation.
LINFLEXD_LIN_IP_STATUS_BUSY	LIN hardware instance is in busy.
LINFLEXD_LIN_IP_STATUS_TIMEOUT	Timeout error.

Definition at line 167 of file Linflexd_Lin_Ip_Types.h.

6.4.5.5 Linflexd_Lin_Ip_FrameCsModelType

```
enum Linflexd_Lin_Ip_FrameCsModelType
```

Enumerator

LINFLEXD_LIN_IP_ENHANCED_CS	Enhanced checksum model.
LINFLEXD_LIN_IP_CLASSIC_CS	Classic checksum model.

Definition at line 175 of file Linflexd_Lin_Ip_Types.h.

6.4.5.6 Linflexd_Lin_Ip_FrameResponseType

```
enum Linflexd_Lin_Ip_FrameResponseType
```

Enumerator

LINFLEXD_LIN_IP_FRAMERESPONSE_TX	Response is generated from this (master) node.
LINFLEXD_LIN_IP_FRAMERESPONSE_RX	Response is generated from a remote slave node.
LINFLEXD_LIN_IP_FRAMERESPONSE_IGNORE	Response is generated from one slave to another slave. For the master the response will be anonymous, it does not have to receive the response.

Definition at line 181 of file Linflexd_Lin_Ip_Types.h.

6.4.6 Function Reference

6.4.6.1 Linflexd_Lin_Ip_Init()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_Init (
    const uint8 Instance,
    const Linflexd_Lin_Ip_UserConfigType * LinUserConfig )
```

Initializes an LIN_LINFLEXD instance for LIN Network.

The caller provides memory for the driver state structures during initialization. The user must select the LIN_LINFLEXD clock source in the application to initialize the LIN_LINFLEXD. This function initializes a LINFLEXD instance for operation. This function will initialize the run-time state structure to keep track of the on-going transfers, initialize the module to user defined settings and default settings, set break field length to be 13 bit times minimum, enable the break detect interrupt, Rx complete interrupt, frame error detect interrupt, and enable the LINFLEXD module transmitter and receiver

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
<i>LinUserConfig</i>	user configuration structure of type #Linflexd_Lin_Ip_UserConfigType

Returns

LINFLEXD_LIN_IP_STATUS_SUCCESS - Initialization command has been accepted. LINFLEXD_LIN_IP_STATUS_ERROR - Initialization command has not been accepted.

6.4.6.2 Linflexd_Lin_Ip_Deinit()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_Deinit (
    const uint8 Instance )
```

Shuts down the LIN_LINFLEXD by disabling interrupts and transmitter/receiver.

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
-----------------	------------------------------

Returns

LINFLEXD_LIN_IP_STATUS_SUCCESS - De-initialization command has been accepted. LINFLEXD_LIN_IP_STATUS_ERROR - De-initialization command has not been accepted.

6.4.6.3 Linflexd_Lin_Ip_SendFrame()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_SendFrame (
    const uint8 Instance,
    const Linflexd_Lin_Ip_PduType * PduInfoPtr )
```

Sends frame out through the LIN_LINFLEXD module using non-blocking method. This enables an a-sync method for transmitting data. Non-blocking means that the function returns immediately. The application has to get the transmit status to know when the transmit is complete. This function will calculate the checksum byte and send it with the frame data after header transmitted. If frame size is equal to 0 or greater than 8 or node's current state is in SLEEP mode then the function will return STATUS_ERROR. If isBusBusy is currently true then the function will return STATUS_BUSY.

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
<i>PduInfoPtr</i>	contains information of frame

Returns

operation status: LINFLEXD_LIN_IP_STATUS_SUCCESS - Send command has been accepted. LINFLEXD_LIN_IP_STATUS_BUSY - Operation failed due to hardware instance busy. LINFLEXD_LIN_IP_STATUS_ERROR - Send command has not been accepted.

6.4.6.4 Linflexd_Lin_Ip_AbortTransferData()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_AbortTransferData (
    const uint8 Instance )
```

Aborts an on-going non-blocking transmission/reception. While performing a non-blocking transferring data, users can call this function to terminate immediately the transferring.

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
-----------------	------------------------------

Returns

operation status: LINFLEXD_LIN_IP_STATUS_SUCCESS - Abort transfer is successful. LINFLEXD_LIN_IP_STATUS_ERROR - Abort transfer is fail.

6.4.6.5 Linflexd_Lin_Ip_GetCurrentNodeState()

```
Linflexd_Lin_Ip_NodeStateType Linflexd_Lin_Ip_GetCurrentNodeState (
    const uint8 Instance )
```

This function is used to retrieve the current state of the lin node.

Parameters

<i>Instance</i>	LIN_LINFLEXD instance number
-----------------	------------------------------

Returns

operation status: Linflexd_Lin_Ip_NodeStateType

6.4.6.6 Linflexd_Lin_Ip_GetStatus()

```
Linflexd_Lin_Ip_TransferStatusType Linflexd_Lin_Ip_GetStatus (
    const uint8 Instance,
    uint8 ** Buffer )
```

This function returns the status of the current non-blocking transfer. If a response reception has been successfully received, Buffer will be referenced to receive buffer.

Parameters

<i>Instance</i>	LIN_LINFLEXD instance number
<i>Buffer</i>	contain received data if reception completed

Returns

operation status: Linflexd_Lin_Ip_TransferStatusType

6.4.6.7 Linflexd_Lin_Ip_GoToSleepMode()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_GoToSleepMode (
    const uint8 Instance )
```

This function puts current node to sleep mode This function changes current node state to LIN_NODE_STATE↵_SLEEP_MODE.

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
-----------------	------------------------------

Returns

operation status: LINFLEXD_LIN_IP_STATUS_SUCCESS - Command has been accepted. LINFLEXD↵_LIN_IP_STATUS_ERROR - Command has not been accepted, error occurred. LINFLEXD_LIN_IP_S↵TATUS_TIMEOUT - Operation has timeout

6.4.6.8 Linflexd_Lin_Ip_GotoIdleState()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_GotoIdleState (
    const uint8 Instance )
```

This function puts current node to idle mode.

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
-----------------	------------------------------

Returns

LINFLEXD_LIN_IP_STATUS_SUCCESS - Command has been accepted. LINFLEXD_LIN_IP_STATUS_ERROR - Command has not been accepted, error occurred.

6.4.6.9 Linflexd_Lin_Ip_SendWakeupSignal()

```
Linflexd_Lin_Ip_StatusType Linflexd_Lin_Ip_SendWakeupSignal (
    const uint8 Instance )
```

Sends a wakeup signal through the LIN_LINFLEXD interface.

Parameters

<i>instance</i>	LIN_LINFLEXD instance number
-----------------	------------------------------

Returns

operation status: LINFLEXD_LIN_IP_STATUS_SUCCESS - Command has been accepted. LINFLEXD_LIN_IP_STATUS_ERROR - Command has not been accepted, error occurred. LINFLEXD_LIN_IP_STATUS_BUSY - Bus busy flag is true.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

